

Action Chunking with Transformers (ACT)

NECKER

6 Settembre 2026

Indice

1	Introduzione: Paradigmi a Confronto (VLA vs ACT)	2
1.1	Differenze Architetture e Compromessi	2
1.2	I Problemi Fisici risolti da ACT	2
2	L'Architettura ACT: Il Flusso Tensoriale End-to-End	3
2.1	Fase 1: Ingestione Sensoriale e Astrazione Visiva (Encoder)	3
2.1.1	A. Il Backbone Visivo (ResNet-18) e le Feature Maps	3
2.1.2	B. Flattening e Iniezione Posizionale 2D	3
2.1.3	C. Propriocezione	3
2.2	Fase 2: CVAE (Solo in Training)	3
2.3	Fondamenti Variazionali in ACT: Spazio Latente, Dinamica delle Gaussiane e Risoluzione della Multimodalità	5
2.3.1	Natura dei Dati: Chunking Temporale e Distribuzioni Locali	6
2.3.2	La Variabile Latente $\mathbf{z}$ e la Necessità della Varianza	6
2.3.3	La Funzione Obiettivo: Bilanciamento tra Ricostruzione e Divergenza KL	6
2.3.4	Disconnessione Strutturale tra Training e Inferenza	7
2.4	Fase 3: Il Motore Generativo (Transformer Encoder-Decoder)	7
2.4.1	Il Transformer Encoder (Costruzione della Memoria Spaziale Implicita)	8
2.4.2	Il Transformer Decoder (Decodifica Parallela delle Traiettorie)	8
3	Action Chunking e Temporal Ensembling	14
3.1	Action Chunking (Mitigazione della miopia temporale)	14
3.2	Temporal Ensembling (Fusione Esponenziale)	14
4	Addestramento: Funzione di Costo e Flusso dei Gradienti	15
4.1	Le due componenti della Loss	15
4.2	Il Paradosso Causale: Perché il CVAE scompare a Runtime?	16

1 Introduzione: Paradigmi a Confronto (VLA vs ACT)

Nel panorama dell'Embodied AI, la ricerca ha sviluppato due filosofie architetture profondamente diverse per risolvere il problema del controllo robotico: i modelli **Vision-Language-Action (VLA)** (come RT-2 o OpenVLA) e i modelli puramente sensomotori come **ACT (Action Chunking with Transformers)**.

Se i VLA simulano la "corteccia prefrontale" del robot (privilegiando il ragionamento semantico astratto), ACT ne rappresenta il "cervelletto", focalizzandosi esclusivamente sulla destrezza fisica, sui riflessi ad alta frequenza e sulla manipolazione millimetrica.

1.1 Differenze Architetture e Compromessi

I modelli VLA utilizzano un Large Language Model (LLM) come motore decisionale: fondono percezione visiva e linguaggio in un unico spazio latente, costringendo il robot a "pensare a parole" prima di agire. Al contrario, ACT abbandona completamente l'LLM. Si affida a un'architettura leggera (pochi milioni di parametri contro i miliardi dei VLA) basata su un **Conditional Variational Autoencoder (CVAE)** e un Transformer Encoder-Decoder standard, che mappa direttamente i pixel e la propriocezione in coordinate cinematiche continue.

Questo disaccoppiamento concettuale genera vantaggi e svantaggi complementari:

Dove i modelli VLA eccellono (I limiti di ACT):

- **Generalizzazione Zero-Shot e Senso Comune:** I VLA ereditano la conoscenza di internet. Sanno a cosa serve un oggetto mai visto prima e sanno eseguire task logicamente complessi (*"Sposta la mela lontano dal bordo del tavolo"*). ACT è "cieco" alla semantica: non sa cosa sia una mela. Impara solo correlazioni geometriche strette dai dati di teleoperazione. Se gli si presenta un oggetto fuori distribuzione (OOD) non presente nel training set, ACT fallisce in modo catastrofico.
- **Ragionamento Emergente:** I VLA possono scomporre istruzioni complesse in step logici e adattarsi a variazioni contestuali, mentre ACT esegue meccanicamente il pattern motorio che ha imparato a memoria.

Dove ACT domina (I limiti dei VLA):

- **Destrezza Fine e Controllo Bimanuale:** I VLA faticano in task di precisione (es. inserire un cavo USB, allacciare scarpe, manipolare oggetti fragili bimanualmente) a causa dell'inferenza lenta e della discretizzazione in token dell'azione. ACT è il *gold standard* per la micro-manipolazione: genera matrici di coordinate continue (non token) a frequenze altissime.
- **Fluidità e Stabilità Dinamica:** I modelli VLA a step singolo soffrono di *Compounding Error* (l'errore di una singola predizione a scatti fa deragliare l'intera traiettoria). ACT risolve questo problema prevedendo interi blocchi di traiettoria futura (*Action Chunking*) e fondendoli nel tempo, garantendo movimenti fisicamente lisci e sicuri.

1.2 I Problemi Fisici risolti da ACT

Nell'ambito dell'Imitation Learning puro (*Behavior Cloning*), ACT è stato progettato specificamente per risolvere l'ostacolo più grande della clonazione del comportamento umano: la **Multimodalità**.

I dimostratori umani sono incoerenti. Di fronte a un ostacolo, un umano può decidere di aggirarlo da destra o da sinistra. Nel dataset avremo la stessa osservazione visiva $\mathbf{o}_t$ associata a due azioni opposte ($\mathbf{a}_{dx}$ e $\mathbf{a}_{sx}$). Minimizzando l'errore classico, una rete neurale deterministica (come i vecchi approcci BC) predice la media matematica delle due traiettorie, portando il robot a schiantarsi dritto contro l'ostacolo.

ACT aggira questo limite usando il **CVAE** per codificare queste variazioni in uno spazio probabilistico, permettendo al robot di campionare uno "stile" coerente di movimento senza collassare in medie fallimentari.

ATTENZIONE: A differenza dei modelli VLA, ACT è una rete puramente geometrica/motoria. Non ragiona attraverso il linguaggio. Il suo obiettivo non è "capire" il mondo, ma tradurre un'immagine e lo stato dei motori in una traiettoria fluida che ricalchi perfettamente la destrezza del suo operatore umano.

2 L'Architettura ACT: Il Flusso Tensoriale End-to-End

Per comprendere appieno la meccanica di **Action Chunking with Transformers (ACT)**, è necessario abbandonare il paradigma dei Large Language Models (basato su sequenze di parole) e ragionare in termini di pura **geometria e percezione spaziale**.

Il modello ACT mappa le matrici di pixel e gli angoli dei giunti direttamente in un campo di traiettorie continue, utilizzando un CVAE per gestire le biforcazioni decisionali (es. ostacolo a destra o a sinistra). Seguiamo il flusso dei tensori dai sensori fisici fino alla predizione dell'azione.

2.1 Fase 1: Ingestione Sensoriale e Astrazione Visiva (Encoder)

L'input istantaneo del robot $\mathbf{o}_t$ è eterogeneo: è composto da flussi video multi-camera e dallo stato meccanico interno.

2.1.1 A. Il Backbone Visivo (ResNet-18) e le Feature Maps

A differenza dei VLA che usano Vision Transformers giganti (es. SigLIP) per estrarre semantica, ACT privilegia la velocità e la localizzazione spaziale. Le immagini in arrivo dalle telecamere (es. $I_{\text{top}}, I_{\text{wrist_left}}, I_{\text{wrist_right}} \in \mathbb{R}^{3 \times 480 \times 640}$) vengono processate da una **CNN standard (ResNet-18)** priva dei layer di classificazione finali.

L'output della ResNet per ogni telecamera non è un singolo vettore, ma una **Feature Map spaziale 2D**:

$$\mathbf{F}_{\text{cam}} \in \mathbb{R}^{C \times H' \times W'} \quad (\text{es. } 512 \times 15 \times 20)$$

Questa griglia tridimensionale preserva la topologia della scena: l'attivazione in alto a sinistra nella mappa corrisponde esattamente all'oggetto in alto a sinistra sul tavolo.

2.1.2 B. Flattening e Iniezione Posizionale 2D

Poiché il Transformer successivo accetta solo sequenze 1D, le mappe di feature vengono "appiattite" (*flattened*) fondendo larghezza e altezza in un'unica sequenza di lunghezza $H' \cdot W'$ (es. 300 token visivi per telecamera).

Per non perdere l'informazione geometrica, a ogni vettore viene sommato un **2D Positional Embedding** (funzioni sinusoidali bidimensionali) che comunica al Transformer le coordinate (x, y) originarie del pixel astratto. Infine, tutti i token visivi delle varie telecamere vengono concatenati in un'unica lunga sequenza.

2.1.3 C. Propriocezione

Parallelamente, lo stato dei giunti $\mathbf{q}_t \in \mathbb{R}^{14}$ (per due braccia a 7 DoF) viene proiettato da un piccolo MLP in un singolo vettore di embedding $\mathbf{e}_{\text{proprio}} \in \mathbb{R}^{d_{\text{model}}}$. Questo vettore viene anteposto alla sequenza visiva.

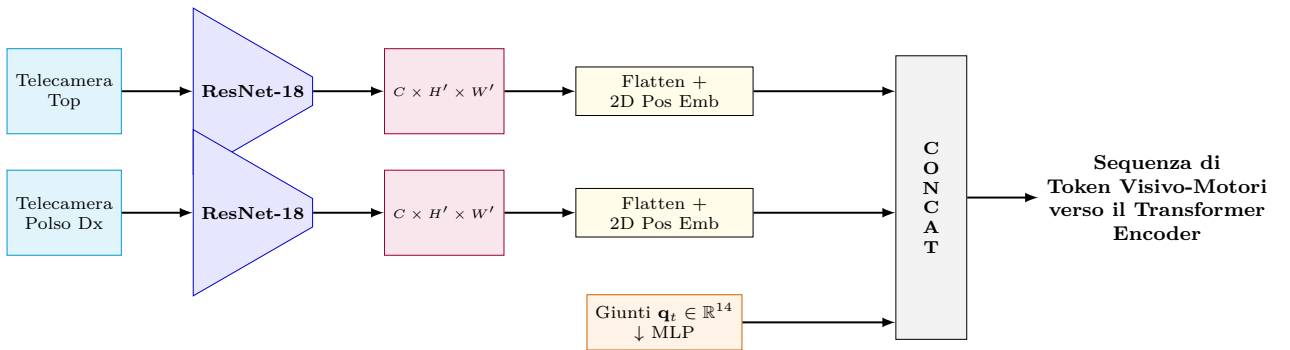


Figura 1: Fase 1: Astrazione e serializzazione dell'osservazione $\mathbf{o}_t$. Le immagini non vengono compresse in un singolo vettore, ma mantengono la loro risoluzione spaziale sotto forma di centinaia di token indipendenti, permettendo al Transformer di focalizzarsi su millimetri specifici della scena.

2.2 Fase 2: CVAE (Solo in Training)

Mentre il Transformer Encoder analizza la scena visiva, il sistema deve gestire la *multimodalità* dell'azione (cioè lo stile dell'azione che viene eseguita poiché la stessa azione può essere fatta in molteplici modi). Questo

avviene unicamente durante l'addestramento, sfruttando il blocco **CVAE Encoder (Conditional Variational Autoencoder)**.

Il CVAE Encoder è un piccolo Transformer che prende in ingresso la **traiettoria reale completa (Ground Truth)** $\mathbf{A}_t \in \mathbb{R}^{H \times D}$ eseguita dall'umano. Il suo compito è leggere l'intero futuro del movimento e riassumere "l'intenzione" in un singolo minuscolo vettore.

1. **Ingestione della Traiettoria:** Fisicamente, la traiettoria di *Ground Truth* $\mathbf{A}_t \in \mathbb{R}^{H \times D}$ non è una linea disegnata nello spazio, ma una vera e propria tabella di numeri: una matrice temporale dove ogni riga rappresenta l'esatto stato dei 14 motori del robot in un preciso centesimo di secondo. Se l'orizzonte è $H = 100$, abbiamo 100 vettori cinematici sequenziali. Questa matrice viene passata attraverso un layer lineare e sommata ai *1D Positional Embeddings*, che fungono da "timestamp" per far capire alla rete l'ordine cronologico esatto dei fotogrammi motori.
2. **Iniezione del Token [CLS]:** In testa a questa sequenza di 100 vettori motori, viene letteralmente incollato un vettore extra (chiamato token [CLS], acronimo di *Classification*). Possiamo immaginarlo come un "foglio bianco" o una "spugna": è un vettore di parametri addestrabili che all'inizio non contiene alcuna coordinata fisica e non è legato a nessun istante di tempo specifico, ma serve solo a raccogliere informazioni.
3. **Self-Attention e Aggregazione Temporale:** I 101 vettori (1 [CLS] + 100 passi motori) attraversano i layer di Self-Attention del Transformer. Poiché il token [CLS] non ha un suo significato predefinito, la matematica dell'attenzione lo forza a interagire con tutti gli altri 100 token, leggendoli simultaneamente. Alla fine del passaggio, il token [CLS] non è più vuoto: ha "assorbito" e sintetizzato il senso globale del movimento. Ad esempio, ha codificato il concetto latente di *"Questa è una manovra fluida di aggiramento"*.
4. **Proiezione Probabilistica:** La rete ora scarta i 100 token temporali e conserva unicamente il token [CLS] arricchito. Questo singolo vettore riassuntivo viene fatto passare attraverso due piccole reti lineari separate per generare i parametri di una distribuzione Gaussiana: un vettore delle medie $\boldsymbol{\mu}$ e un vettore delle varianze $\boldsymbol{\sigma}^2$.
5. **Campionamento e Riparametrizzazione:** Infine, si estrae il vettore di stile $\mathbf{z} \in \mathbb{R}^{d_z}$ campionando da questa distribuzione. Per permettere al gradiente (Backpropagation) di attraversare un'operazione casuale senza bloccarsi, si usa il *Reparameterization Trick*:

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \quad \text{con } \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Il vettore $\mathbf{z}$ risultante è la versione ultracompressa (es. soli 32 numeri) dell'intera matrice di traiettoria iniziale 100×14 .

Esempio Intuitivo: Gestire la Multimodalità con il [CLS] Token Immaginiamo che il robot debba afferrare una mela, ma in mezzo al tavolo c'è una bottiglia. Durante la raccolta dati, il dimostratore umano a volte aggira la bottiglia da **destra**, a volte da **sinistra** (questa è la *multimodalità* del comportamento umano).

Se diamo al CVAE Encoder la matrice $\mathbf{A}_t$ di una dimostrazione in cui l'umano è passato da **sinistra**:

- La matrice contiene i 100 fotogrammi angolari che piegano progressivamente il braccio a sinistra.
- Il token [CLS] "osserva" tutti questi 100 fotogrammi e ne astrae l'intenzione: *"I motori della spalla stanno deviando lungo l'asse X negativo per poi raddrizzarsi"*.
- Il token viene trasformato in $\boldsymbol{\mu}$ e $\boldsymbol{\sigma}$, dai quali viene estratto $\mathbf{z}$.
- Matematicamente, $\mathbf{z}$ diventerà un vettore di 32 numeri che risiede in una specifica regione dello spazio latente (es. l'emisfero sinistro della Gaussiana), codificando la decisione *"Stile di evitamento a sinistra"*.

Quando questo $\mathbf{z}$ verrà successivamente iniettato nel Transformer Decoder, agirà come un "comando occulto": forzerà il Decoder a generare unicamente coordinate per passare a sinistra, impedendogli di mediare ciecamente le due opzioni e finire per schiantarsi dritto contro la bottiglia centrale.

Nota cruciale: A inferenza, questo intero ramo viene spento (poiché non conosciamo il futuro $\mathbf{A}_t$). Il vettore $\mathbf{z}$ viene semplicemente campionato da una Gaussiana standard, o impostato a $\mathbf{0}$ per il comportamento più probabile (deterministico).

ATTENZIONE: ogni gaussiana rappresenta uno stile specifico di una frazione di azione che deve fare il robot,

tutte le gaussiane sono spinte da **KL** a posizionarsi vicine nello spazio latente, z rappresenta l'azione specifica campionata (la quale può variare leggermente dato che è estratta da una delle gaussiane).

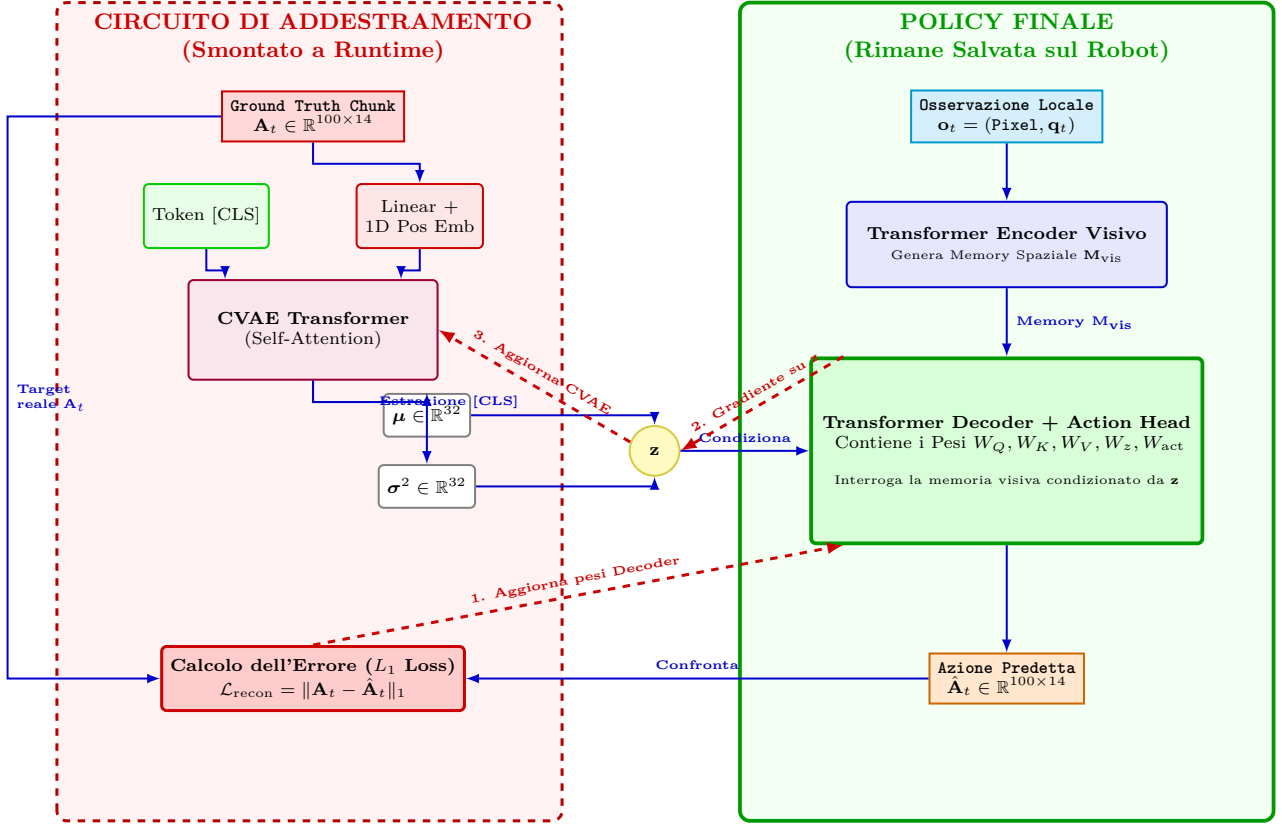


Figura 2: Architettura dettagliata e ciclo di co-addestramento in ACT. Il riquadro sinistro (attivo esclusivamente durante il training) espande la meccanica interna del CVAE: il chunk futuro viene aggregato dal token [CLS] e processato dal Transformer per ricavare la distribuzione gaussiana (μ, σ^2) . Da essa viene campionato lo stile z , che condiziona il Decoder (riquadro destro). Durante la backpropagation (freccie rosse tratteggiate), l'errore L_1 aggiorna i pesi del Decoder e risale a ritroso per calibrare l'Encoder. A runtime, l'intero riquadro sinistro viene isolato e disattivato.

2.3 Fondamenti Variazionali in ACT: Spazio Latente, Dinamica delle Gaussiane e Risoluzione della Multimodalità

Una delle criticità fondamentali nell'apprendimento per imitazione da dimostrazioni umane (*Behavioral Cloning*) risiede nella **multimodalità intrinseca** dei dati di teleoperazione: a parità di osservazione visuo-proprioceettiva $\mathbf{o}_t$, operatori umani differenti (o lo stesso operatore in istanti distinti) possono compiere scelte cinematiche radicalmente divergenti (es. aggirare un ostacolo passando da destra o da sinistra).

Se un modello deterministico standard viene ottimizzato mediante una perdita di regressione classica (es. errore L_1 o L_2) direttamente sullo spazio delle azioni $\mathbf{A}_t$, la minimizzazione dell'errore atteso forza i parametri della rete a convergere sulla *media geometrica* delle dimostrazioni:

$$\hat{\mathbf{A}}_{\text{collasso}} \approx \mathbb{E}[\mathbf{A}_t | \mathbf{o}_t] = \frac{\mathbf{A}_{\text{sinistra}} + \mathbf{A}_{\text{destra}}}{2}$$

Nel mondo fisico, la traiettoria media risultante guida l'organo terminale in rotta di collisione frontale con l'ostacolo centrale, rendendo la politica inutilizzabile. Per isolare e disaccoppiare queste modalità, ACT implementa un'architettura basata su un **Conditional Variational Autoencoder** (CVAE).

2.3.1 Natura dei Dati: Chunking Temporale e Distribuzioni Locali

Una distribuzione gaussiana nel CVAE non modella l'intero episodio operativo (della durata di decine di secondi), bensì un singolo **Action Chunk** locale (una frazione di movimento):

$$\mathbf{A}_t = \begin{bmatrix} \mathbf{a}_t \\ \mathbf{a}_{t+1} \\ \vdots \\ \mathbf{a}_{t+H-1} \end{bmatrix} \in \mathbb{R}^{H \times D}$$

Dato un dataset di teleoperazione a frequenza di campionamento costante (es. 50 Hz), una finestra temporale mobile di orizzonte H (es. $H = 100$ passi, equivalenti a 2 s) scorre lungo ciascun episodio dimostrativo. Un corpus composto da centinaia di episodi genera centinaia di migliaia di coppie di addestramento $(\mathbf{o}_t, \mathbf{A}_t)$. Ciascuna di queste finestre temporali viene mappata in una propria distribuzione condizionale locale.

2.3.2 La Variabile Latente $\mathbf{z}$ e la Necessità della Varianza

L'Encoder del CVAE comprime la finestra temporale locale dell'action chunk $\mathbf{A}_t$ (i soli H passi futuri, tipicamente 1–2 secondi, e non l'intero episodio globale del task) in una variabile latente a bassa dimensionalità:

$$\mathbf{z} \in \mathbb{R}^{d_z} \quad (\text{tipicamente } d_z = 32)$$

$\mathbf{z}$ rappresenta un vettore di coordinate continue che formalizza lo "stile" o la specifica declinazione cinematica limitata a questa singola frazione di movimento (es. curvatura locale dell'approccio, profilo di velocità contingente, apertura angolare preventiva del gripper).

A differenza di un Autoencoder deterministico convenzionale, l'Encoder non emette un singolo vettore discreto $\mathbf{z}$, ma mappa l'input nei parametri di una distribuzione gaussiana multivariata a matrice di covarianza diagonale:

$$\boldsymbol{\mu}_t, \boldsymbol{\sigma}_t = \text{Encoder}_{\text{CVAE}}(\mathbf{A}_t, \mathbf{o}_t)$$

La presenza esplicita della deviazione standard $\boldsymbol{\sigma}_t$ è una condizione strutturale irrinunciabile:

- **Prevenzione dello Spazio Latente Vuoto (*Hole Problem*):** Se l'Encoder proiettasse ogni chunk in un punto deterministico isolato ($\boldsymbol{\sigma} = \mathbf{0}$), lo spazio latente risulterebbe costituito da singolarità sparse immerse in un dominio indefinito. Qualsiasi punto non campionato esattamente durante il training farebbe divergere l'attivazione dei layer del Decoder, innescando oscillazioni incontrollate dei motori.
- **Robustezza di Intorno (*Ball of Tolerance*):** Il campionamento stocastico mediante il *Reparameterization Trick*:

$$\mathbf{z} = \boldsymbol{\mu}_t + \boldsymbol{\sigma}_t \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

perturba costantemente la rappresentazione latente durante la discesa di gradiente. Il Decoder è costretto ad apprendere che l'intero volume iper-sferico di raggio proporzionale a $\boldsymbol{\sigma}_t$ attorno al centroide $\boldsymbol{\mu}_t$ mappa univocamente verso la medesima classe di traiettorie coerenti.

2.3.3 La Funzione Obiettivo: Bilanciamento tra Ricostruzione e Divergenza KL

La continuità e la densità dello spazio latente derivano dall'equilibrio analitico dell'**Evidence Lower Bound** (ELBO), minimizzato durante la fase di training:

$$\mathcal{L}_{\text{CVAE}} = \mathcal{L}_{\text{reconstruction}} + \beta \cdot \mathcal{D}_{\text{KL}}(q_\phi(\mathbf{z} \mid \mathbf{A}_t, \mathbf{o}_t) \parallel p(\mathbf{z}))$$

dove la perdita di ricostruzione è tipicamente calcolata come norma L_1 tra le coordinate target e le predizioni dell'Action Head:

$$\mathcal{L}_{\text{reconstruction}} = \frac{1}{H \cdot D} \sum_{i=1}^H \sum_{j=1}^D |a_{t+i-1,j} - \hat{a}_{t+i-1,j}|$$

Il termine di regolarizzazione impone la **Divergenza di Kullback-Leibler** rispetto a una distribuzione prior gaussiana standard isotropa $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$:

$$\mathcal{D}_{\text{KL}}(q_\phi(\mathbf{z} \mid \mathbf{A}_t, \mathbf{o}_t) \parallel \mathcal{N}(\mathbf{0}, \mathbf{I})) = -\frac{1}{2} \sum_{k=1}^{d_z} (1 + \log(\sigma_{t,k}^2) - \mu_{t,k}^2 - \sigma_{t,k}^2)$$

Questo termine agisce come una forza centripeta fondamentale:

1. **Compressione Centrata** ($\mu_{t,k}^2 \rightarrow 0$): Penalizza la dispersione dei centroidi, forzando tutti i cluster di stile a gravitare all'interno della sfera unitaria centrata nell'origine.
2. **Rigetto del Collasso Deterministico** ($\sigma_{t,k}^2 \rightarrow 1$): Impedisce che la varianza si annulli, poiché per $\sigma_{t,k}^2 \rightarrow 0$ il termine logaritmico $\log(\sigma_{t,k}^2) \rightarrow -\infty$, facendo divergere la funzione di perdita a $+\infty$.

Il bilanciamento guidato dal peso β assicura che le diverse distribuzioni locali non si allontanino eccessivamente né si sovrappongano fino al punto di annullare la specificità dello stile (*Posterior Collapse*). Lo spazio latente risultante assume la struttura di un **manifold compatto e continuo**: lungo qualsiasi percorso rettilineo di interpolazione $\mathbf{z}(\alpha) = (1 - \alpha)\mathbf{z}_A + \alpha\mathbf{z}_B$, le traiettorie motorie generate dal Decoder variano in modo liscio e differenziabile, senza salti dimensionali o singolarità.

2.3.4 Disconnessione Strutturale tra Training e Inferenza

Un'ambiguità comune nell'analisi di ACT riguarda il modo in cui il robot seleziona lo stile a tempo di esecuzione (*runtime*). È fondamentale evidenziare che **a runtime non avviene alcuna ricerca, confronto o calcolo della gaussiana più vicina**.

Durante l'inferenza sul manipolatore reale, la traiettoria futura $\mathbf{A}_t$ è per definizione sconosciuta; di conseguenza, il blocco $\text{Encoder}_{\text{CVAE}}$ viene **completamente disconnesso e rimosso dalla memoria operativa**. Il modello opera secondo due modalità:

- **Inferenza Deterministica Nominale** ($\mathbf{z} = \mathbf{0}$): Nella quasi totalità delle implementazioni pratiche ad anello chiuso, il vettore latente viene impostato permanentemente sul vettore nullo:

$$\mathbf{z}_{\text{runtime}} = [0, 0, \dots, 0]^T \in \mathbb{R}^{d_z}$$

Poiché la Divergenza KL ha compattato la totalità dei modi operativi attorno all'origine della Gaussiana standard, l'origine $\mathbf{z} = \mathbf{0}$ coincide con la modalità nominale a più alta densità di probabilità congiunta. L'eliminazione della componente stocastica sopprime ogni fonte di jitter articolare non deterministico, producendo movimenti fluidi e ripetibili.

- **Inferenza Stocastica Esplorativa** ($\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$): Qualora il robot incorra in un fallimento di presa o in una condizione di stallo cinematico, la variabile $\mathbf{z}$ può essere ricampionata direttamente dal prior standard, consentendo alla politica di istanziare una traiettoria alternativa appartenente a un ramo modale differente.

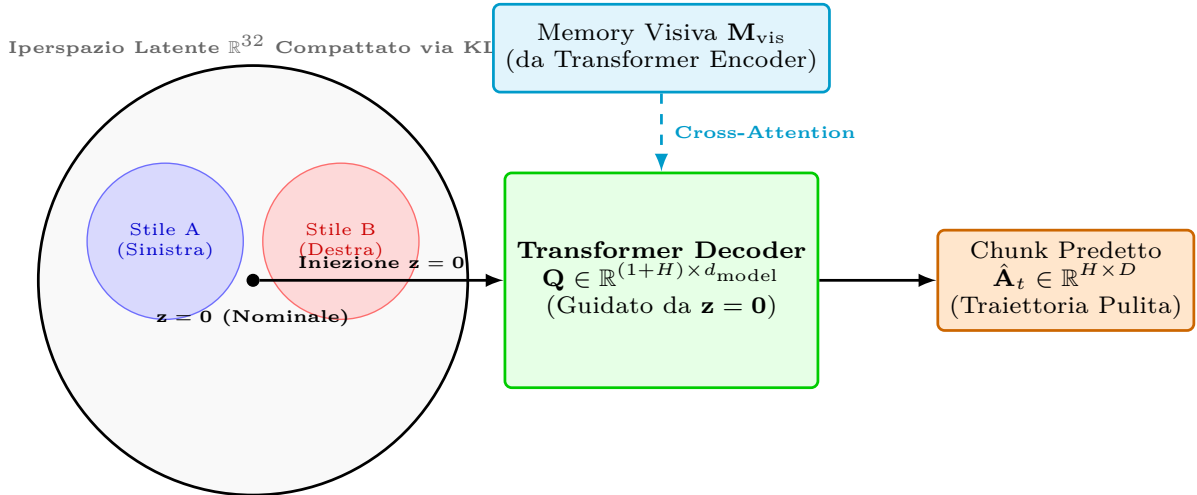


Figura 3: Operatività a runtime del CVAE in ACT. L'Encoder CVAE viene escluso. Fissando $\mathbf{z} = \mathbf{0}$ sull'origine dello spazio latente regolarizzato, il Transformer Decoder sfrutta la Cross-Attention sulla sola memoria percettiva $\mathbf{M}_{\text{vis}}$ per dirimere la traiettoria nominale ottimale priva di collasso sulla media.

2.4 Fase 3: Il Motore Generativo (Transformer Encoder-Decoder)

La *Policy* di ACT sintetizza le coordinate cinematiche attuabili $\hat{\mathbf{A}}_t$ fondendo la rappresentazione geometrico-spaziale dell'ambiente (derivata dall'osservazione $\mathbf{o}_t$) con il vincolo modale imposto dalla variabile latente di stile $\mathbf{z}$.

A differenza dei modelli linguistici causali o autoregressivi, il generatore non produce le azioni sequenzialmente condizionandosi sui propri output passati: mutua invece l'architettura di **DETR** (*DEtection TRansformer*), risolvendo la generazione del chunk temporale come un problema di predizione parallela e congiunta mediante un insieme fisso di vettori di interrogazione (*Queries*).

2.4.1 Il Transformer Encoder (Costruzione della Memoria Spaziale Implicita)

L'input dell'Encoder è la matrice dei token visuo-proprioceettivi assemblata al timestep corrente:

$$\mathbf{X}_{\text{vis}} = \begin{bmatrix} \mathbf{x}_1 \\ \vdots \\ \mathbf{x}_{N_{\text{tokens}}} \end{bmatrix} \in \mathbb{R}^{N_{\text{tokens}} \times d_{\text{model}}}$$

dove ciascuna riga rappresenta un elemento locale (un patch 2D estratto dai feature map convoluzionali delle telecamere o lo stato scalare dei giunti $\mathbf{q}_t$, proiettati a $d_{\text{model}} = 512$ e sommati ai rispettivi embedding posizionali 2D/1D) come spiegato in precedenza.

All'interno di ciascuno degli L_{enc} strati dell'Encoder, la matrice viene proiettata tramite tre matrici di peso addestrabili $W_Q^{\text{enc}}, W_K^{\text{enc}}, W_V^{\text{enc}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$:

$$Q = \mathbf{X}W_Q^{\text{enc}}, \quad K = \mathbf{X}W_K^{\text{enc}}, \quad V = \mathbf{X}W_V^{\text{enc}}$$

Scomponendo la computazione in N_h teste di attenzione parallele ($d_k = d_{\text{model}}/N_h = 512/8 = 64$ dimensioni per testa), il coefficiente di attenzione scalare tra il token generico i e il token generico j vale:

$$\alpha_{ij} = \frac{\exp\left(\frac{\mathbf{q}_i \mathbf{k}_j^T}{\sqrt{d_k}}\right)}{\sum_{l=1}^{N_{\text{tokens}}} \exp\left(\frac{\mathbf{q}_i \mathbf{k}_l^T}{\sqrt{d_k}}\right)}$$

- **Meccanica di Correlazione Visiva:** Se il token i rappresenta un patch della telecamera zenitale inquadrante il bordo di una tazza e il token j proviene dalla telecamera al polso che inquadra il manico, l'ottimizzazione minimizza la distanza angolare tra $\mathbf{q}_i$ e $\mathbf{k}_j$. Il prodotto scalare $\mathbf{q}_i \mathbf{k}_j^T$ cresce, producendo un peso di attenzione α_{ij} dominante.
- **Fusione Topologica:** Il nuovo vettore del token i viene aggiornato come combinazione convessa:

$$\mathbf{x}'_i = \sum_{j=1}^{N_{\text{tokens}}} \alpha_{ij} \mathbf{v}_j$$

integrando l'informazione visiva sparsa di punti di vista differenti all'interno di un unico frame di riferimento coordinato.

Dopo il passaggio attraverso connessioni residue, *Layer Normalization* e una rete *Feed-Forward* densa bidirezionale a due strati per ciascun layer:

$$\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x}W_1^{\text{enc}} + \mathbf{b}_1^{\text{enc}}) W_2^{\text{enc}} + \mathbf{b}_2^{\text{enc}}, \quad W_1^{\text{enc}} \in \mathbb{R}^{512 \times 2048}, W_2^{\text{enc}} \in \mathbb{R}^{2048 \times 512}$$

L'Encoder emette la matrice finale non modificabile:

$$\mathbf{M}_{\text{vis}} \in \mathbb{R}^{N_{\text{tokens}} \times d_{\text{model}}}$$

denominata **Memory**. Questa matrice non contiene più vettori di pixel isolati, ma una mappa semantica e volumetrica continua della scena operativa contingente.

ATTENZIONE: come studiato in altri contesti, il modulo di attenzione ha lo scopo di preparare i dati, la rete **FFN** invece ha lo scopo di analizzarli.

2.4.2 Il Transformer Decoder (Decodifica Parallela delle Traiettorie)

Il Decoder ha il compito di interrogare la *Memory* $\mathbf{M}_{\text{vis}}$ per generare le coordinate fisiche lungo l'intero orizzonte temporale H in un singolo passo computazionale. Il flusso analitico si articola in quattro stadi sequenziali:

1. Composizione della Matrice delle Query ($\mathbf{Q}_{\text{in}}$) Il Decoder non riceve in ingresso dati sensoriali grezzi, ma una sequenza strutturata di interrogazioni latenti:

- **Iniezione di Stile come Token 0:** Il vettore $\mathbf{z} \in \mathbb{R}^{32}$ (estratto dalla distribuzione variazionale durante l'addestramento, oppure impostato deterministicamente sul vettore nullo $\mathbf{z} = \mathbf{0}_{32}$ a runtime) viene proiettato nello spazio interno del modello mediante la matrice di proiezione lineare $W_z \in \mathbb{R}^{32 \times 512}$ e il bias $\mathbf{b}_z \in \mathbb{R}^{512}$:

$$\mathbf{e}_z = \mathbf{z}W_z + \mathbf{b}_z \in \mathbb{R}^{1 \times 512}$$

- **Positional Queries Temporali:** Vengono istanziati H vettori di query temporale $\mathbf{p}_1, \dots, \mathbf{p}_H \in \mathbb{R}^{1 \times 512}$. Questi parametri sono vettori addestrabili (*learned query embeddings*) che non dipendono dallo stato percettivo del robot; ciascuno codifica formalmente la coordinata temporale astratta dell'orizzonte:

$$\mathbf{p}_h \equiv \text{"Configurazione cinematica target per l'istante relativo } t + h \cdot \Delta t\text{"}$$

I vettori vengono impilati lungo la dimensione sequenziale ($N_{\text{dec}} = 1 + H$ token):

$$\mathbf{Q}_{\text{in}} = \begin{bmatrix} \mathbf{e}_z \\ \mathbf{p}_1 \\ \mathbf{p}_2 \\ \vdots \\ \mathbf{p}_H \end{bmatrix} \in \mathbb{R}^{(1+H) \times 512}$$

2. Self-Attention nel Decoder (Coerenza Trajectory-Level) Nel primo blocco di ogni layer del Decoder, $\mathbf{Q}_{\text{in}}$ attraversa una Multi-Head Self-Attention non causale (bidirezionale):

$$Q_{\text{self}} = \mathbf{Q}_{\text{in}} W_{Q,\text{self}}^{\text{dec}}, \quad K_{\text{self}} = \mathbf{Q}_{\text{in}} W_{K,\text{self}}^{\text{dec}}, \quad V_{\text{self}} = \mathbf{Q}_{\text{in}} W_{V,\text{self}}^{\text{dec}}$$

- Poiché non vi è alcuna maschera triangolare inferiore, la query dell'istante $\mathbf{p}_h$ calcola il prodotto scalare simultaneamente con il token di stile $\mathbf{e}_z$ e con tutte le altre query temporali $\mathbf{p}_k$ ($k \neq h$).
- **Diffusione dello Stile:** L'attenzione tra la riga temporale generica h e la riga 0 estrae e incorpora la componente direzionale di $\mathbf{e}_z$, imponendo che la scelta modale (es. aggiramento ad ampio raggio) condizioni uniformemente tutti i passi temporali.
- **Continuità Cinematica:** L'attenzione incrociata tra $\mathbf{p}_h$ e $\mathbf{p}_{h+1}$ vincola la derivata temporale tra passi successivi, prevenendo variazioni d'angolo che eccedano i limiti fisici di velocità o accelerazione articolare.

3. Cross-Attention sulla Memoria Spaziale (Interrogazione della Scena) La matrice delle query processata dalla Self-Attention, denotata con $\mathbf{U} \in \mathbb{R}^{(1+H) \times 512}$, costituisce le Query della Cross-Attention. Le Chiavi e i Valori sono invece derivati dalla memoria percettiva $\mathbf{M}_{\text{vis}}$:

$$Q_{\text{cross}} = \mathbf{U} W_{Q,\text{cross}}^{\text{dec}} \in \mathbb{R}^{(1+H) \times 512}, \quad K_{\text{cross}} = \mathbf{M}_{\text{vis}} W_{K,\text{cross}}^{\text{dec}} \in \mathbb{R}^{N_{\text{tokens}} \times 512}, \quad V_{\text{cross}} = \mathbf{M}_{\text{vis}} W_{V,\text{cross}}^{\text{dec}} \in \mathbb{R}^{N_{\text{tokens}} \times 512}$$

Il calcolo dei coefficienti modula l'estrazione visiva:

$$\mathbf{A}_{\text{cross}} = \text{Softmax} \left(\frac{Q_{\text{cross}} K_{\text{cross}}^T}{\sqrt{d_k}} \right) \in \mathbb{R}^{(1+H) \times N_{\text{tokens}}}$$

Ciascuna riga $h \in \{1, \dots, H\}$ di questa matrice di attenzione rappresenta una distribuzione di probabilità sull'intero volume di token visivi e propriocettivi:

- Per $h = 1$ ($t + 20$ ms), il vettore delle query interroga primariamente i token di $\mathbf{M}_{\text{vis}}$ corrispondenti allo stato corrente del robot e all'ingombro immediato della pinza.
- Per $h = H$ (orizzonte temporale finale), la query proietta i pesi di attenzione sui token di $\mathbf{M}_{\text{vis}}$ che codificano il target finale dell'interazione (ad esempio il piano d'appoggio o la regione di rilascio del manipolando).

Il prodotto $\mathbf{A}_{\text{cross}} V_{\text{cross}}$ genera una matrice in $\mathbb{R}^{(1+H) \times 512}$, dove ogni passo futuro ha estratto i soli vincoli geometrici necessari alla propria attuazione.

4. Estrazione Cinematica e Action Head Dopo il transito attraverso la rete feed-forward interna al Decoder, la riga 0 (il token originato dalla proiezione di $\mathbf{z}$) viene scartata, avendo esaurito la propria funzione strutturale di polarizzazione dei layer interni:

$$\mathbf{H}_{\text{out}} = \mathbf{Y}_{\text{dec}}[1 : H + 1, :] \in \mathbb{R}^{H \times 512}$$

La matrice $\mathbf{H}_{\text{out}}$ entra nell'**Action Head**, implementata in questo tracciamento come una proiezione lineare parallela su ciascuna delle H righe:

$$\hat{\mathbf{A}}_t = \mathbf{H}_{\text{out}} \mathbf{W}_{\text{act}} + \mathbf{b}_{\text{act}}$$

con pesi dimensionali:

$$\mathbf{W}_{\text{act}} \in \mathbb{R}^{512 \times D}, \quad \mathbf{b}_{\text{act}} \in \mathbb{R}^D$$

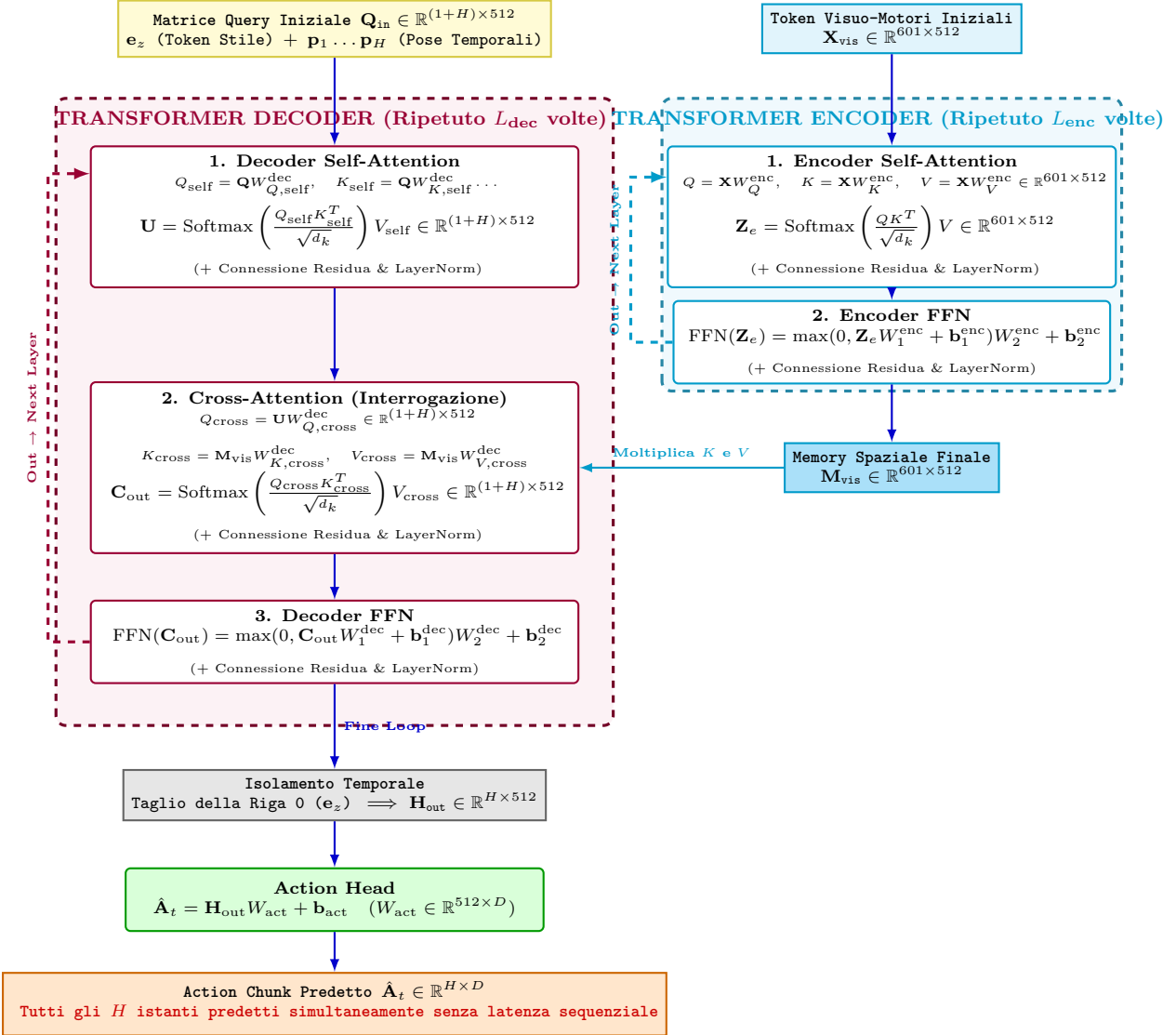


Figura 4: Decostruzione matematica profonda del Motore Generativo. A destra l'Encoder Visivo: fonde iterativamente i pixel grezzi sfruttando la Self-Attention per mappare le correlazioni geometriche tra telecamere e la FFN per estrarne il significato, producendo la Memory $\mathbf{M}_{\text{vis}}$. A sinistra il Decoder: la matrice temporale delle query si stabilizza internamente (Self-Attention) e proietta i propri $\mathbf{Q}_{\text{cross}}$ sui $\mathbf{K}_{\text{cross}}$ visivi. Il risultato finale attraversa l'Action Head emettendo l'intera cinematica futura in parallelo.

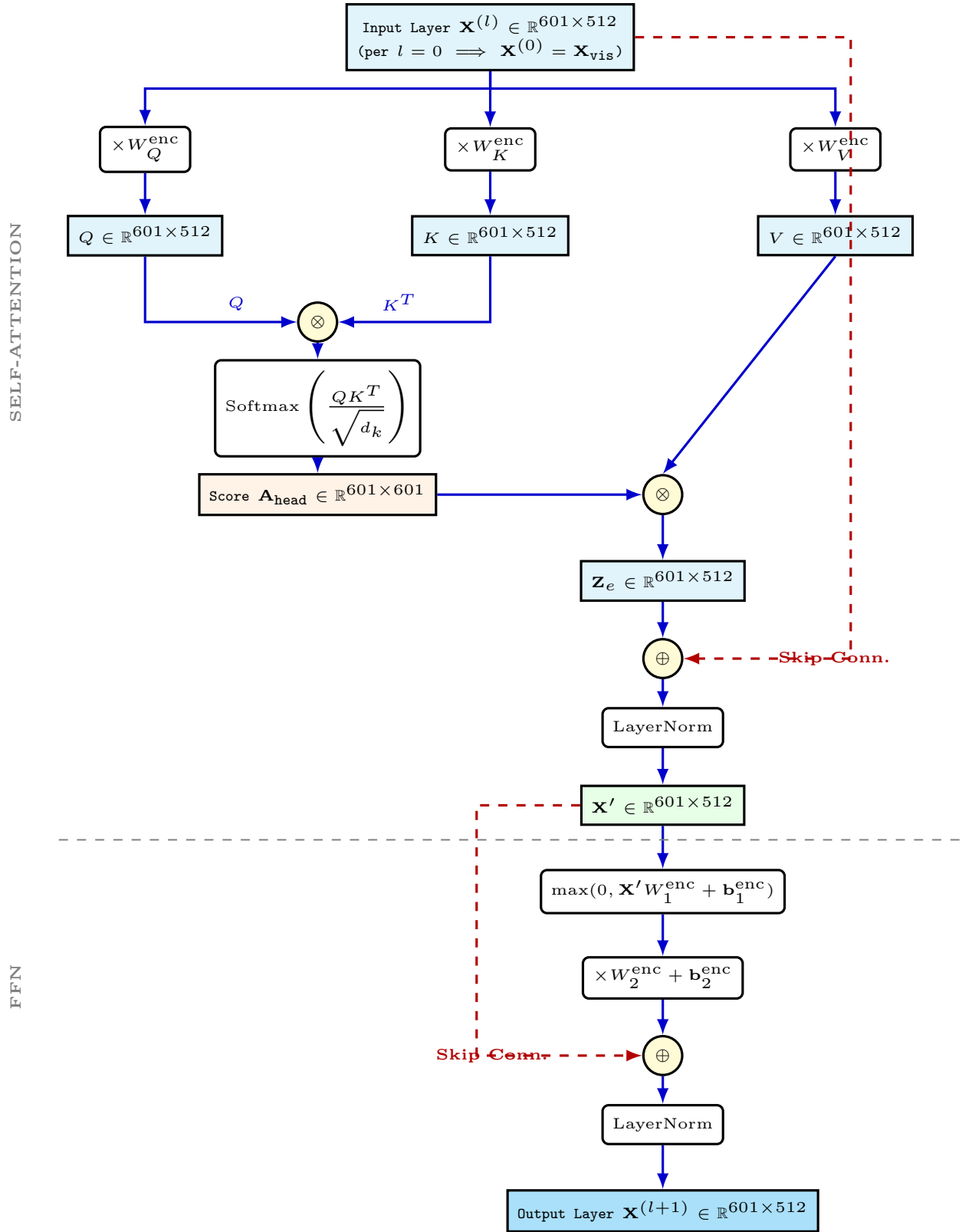


Figura 5: Diagramma di flusso computazionale compatto di un singolo strato del Transformer Encoder. Ogni formula è srotolata nei suoi passaggi tensoriali: la scomposizione in Query/Key/Value mediante proiezioni $W_{\{Q,K,V\}}^{\text{enc}}$, il prodotto matriciale ($\otimes$) per calcolare gli *attention scores* $\mathbf{A}_{\text{head}}$, la combinazione convessa con il Valore e l'attraversamento della FFN non-lineare.

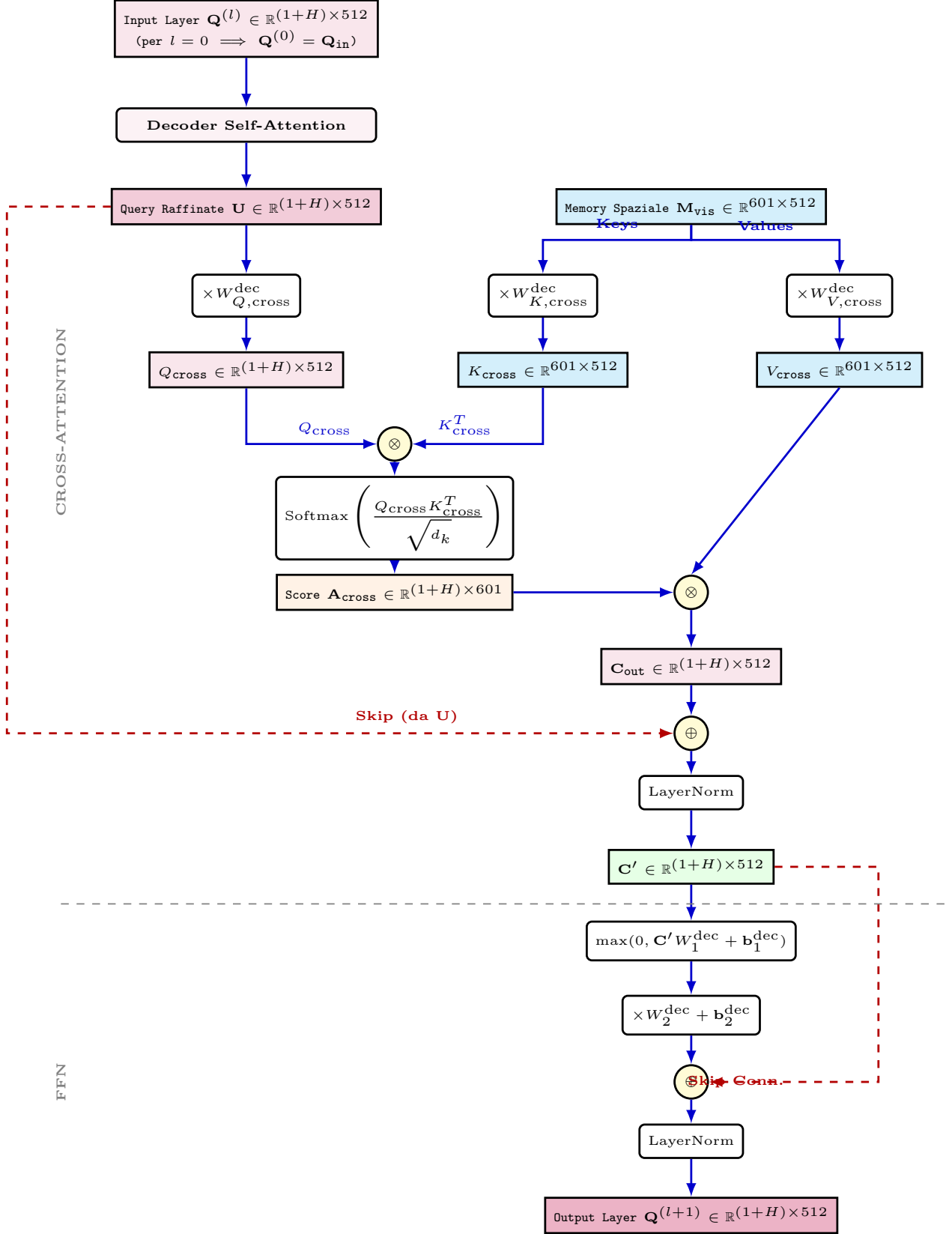


Figura 6: Scomposizione matematica compatta della Cross-Attention nel Transformer Decoder. Le *Queries* $\mathbf{Q}_{\text{cross}}$ derivano dalla sequenza temporale interna $\mathbf{U}$, mentre *Keys* $\mathbf{K}_{\text{cross}}$ e *Values* $\mathbf{V}_{\text{cross}}$ originano dalla memoria percettiva $\mathbf{M}_{\text{vis}}$. Il prodotto normalizzato determina la matrice $\mathbf{A}_{\text{cross}}$, che estrae e aggrega le feature rilevanti in $\mathbf{C}_{\text{out}}$ per ciascun timestep futuro.

Tracciamento Numerico Rigoroso: Tensori e Connessioni a Livello Neuronale Fissiamo i parametri standard di una cella bimanuale ad alte prestazioni:

- Orizzonte temporale di predizione: $H = 4$ timesteps (80 ms a 50 Hz).
- Gradi di libertà: $D = 14$ (6 assi cinematici +1 asse gripper per ciascun braccio).
- Dimensione del modello: $d_{\text{model}} = 512$, ripartito su $N_h = 8$ teste di attenzione ($d_k = 64$).
- Dimensione della scena visiva: $N_{\text{tokens}} = 601$ (3 telecamere da 200 patch ciascuna +1 token per la proiezione corrente $\mathbf{q}_t$).

Tracciamo il transito analitico dei numeri e la dimensionalità di ogni singola matrice di peso:

1. **Memory Spaziale dell'Encoder:** La sequenza d'ingresso $\mathbf{X}_{\text{vis}} \in \mathbb{R}^{601 \times 512}$ attraversa la Self-Attention dell'Encoder. L'operazione matriciale:

$$\mathbf{M}_{\text{vis}} = \text{LayerNorm} \left(\text{Softmax} \left(\frac{(\mathbf{X}W_Q^{\text{enc}})(\mathbf{X}W_K^{\text{enc}})^T}{\sqrt{64}} \right) (\mathbf{X}W_V^{\text{enc}}) + \mathbf{X} \right) \in \mathbb{R}^{601 \times 512}$$

stabilizza la rappresentazione. I $601 \times 512 = 307.712$ valori scalari contengono le coordinate metriche estrinseche latenti dell'ambiente, dove ogni vettore riga esprime le correlazioni d'intersezione tra le viste ottiche.

2. **Iniezione di Stile ed Espansione delle Query:** A runtime, il controllore istanzia il vettore nullo deterministico $\mathbf{z} = [0, 0, \dots, 0]^T \in \mathbb{R}^{1 \times 32}$. La proiezione lineare:

$$\mathbf{e}_z = \mathbf{z}W_z + \mathbf{b}_z = [0, \dots, 0] \begin{bmatrix} w_{1,1} & \dots & w_{1,512} \\ \vdots & \ddots & \vdots \\ w_{32,1} & \dots & w_{32,512} \end{bmatrix} + \mathbf{b}_z = \mathbf{b}_z \in \mathbb{R}^{1 \times 512}$$

estorce unicamente il vettore di polarizzazione $\mathbf{b}_z$ calibrato dalla backpropagation per rappresentare il centro geometrico dello spazio latente regolarizzato dal CVAE.

Le $H = 4$ query temporali $\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3, \mathbf{p}_4 \in \mathbb{R}^{1 \times 512}$ vengono concatenate a $\mathbf{e}_z$:

$$\mathbf{Q}_{\text{in}} = \begin{bmatrix} \mathbf{e}_z \\ \mathbf{p}_1 \\ \mathbf{p}_2 \\ \mathbf{p}_3 \\ \mathbf{p}_4 \end{bmatrix} = \begin{bmatrix} e_{z,1} & e_{z,2} & \dots & e_{z,512} \\ p_{1,1} & p_{1,2} & \dots & p_{1,512} \\ p_{2,1} & p_{2,2} & \dots & p_{2,512} \\ p_{3,1} & p_{3,2} & \dots & p_{3,512} \\ p_{4,1} & p_{4,2} & \dots & p_{4,512} \end{bmatrix} \in \mathbb{R}^{5 \times 512}$$

3. **Accoppiamento di Cross-Attention:** Le Query raffinate internamente tramite Self-Attention ($\mathbf{U} \in \mathbb{R}^{5 \times 512}$) e la Memory visiva ($\mathbf{M}_{\text{vis}} \in \mathbb{R}^{601 \times 512}$) generano i tensori di Cross-Attention:

$$\mathbf{Q}_{\text{cross}} = \mathbf{U}W_{Q,\text{cross}}^{\text{dec}} \in \mathbb{R}^{5 \times 512}, \quad \mathbf{K}_{\text{cross}} = \mathbf{M}_{\text{vis}}W_{K,\text{cross}}^{\text{dec}} \in \mathbb{R}^{601 \times 512}$$

Il prodotto scalare per ciascuna testa ($d_k = 64$) produce una matrice di score:

$$\mathbf{S}_{\text{head}} = \frac{\mathbf{Q}_{\text{cross}}\mathbf{K}_{\text{cross}}^T}{\sqrt{64}} \in \mathbb{R}^{5 \times 601}$$

Applicando l'operatore Softmax riga per riga:

$$\mathbf{A}_{\text{head}}[i, j] = \frac{\exp(\mathbf{S}_{\text{head}}[i, j])}{\sum_{k=1}^{601} \exp(\mathbf{S}_{\text{head}}[i, k])}$$

La riga $i = 2$ (associata al timestep $t + 1$) attribuisce pesi superiori (es. $\mathbf{A}[2, 42] = 0.38$) ai token visivi indicanti lo stato istantaneo della pinza, mentre la riga $i = 5$ (associata al timestep $t + 4$) distribuisce la sua intensità focale (es. $\mathbf{A}[5, 310] = 0.45$) sui token indicanti l'oggetto target sul tavolo.

La moltiplicazione per i Valori $V_{\text{cross}} \in \mathbb{R}^{601 \times 512}$ aggrega l'informazione:

$$\mathbf{C}_{\text{out}} = \mathbf{A}_{\text{cross}}V_{\text{cross}} \in \mathbb{R}^{5 \times 512}$$

4. **Proiezione Continua dei Comandi Fisici:** All'uscita dell'ultimo layer di decodifica, si elimina la riga di condizionamento dello stile (riga 0), isolando la matrice contenente i soli vettori temporali evoluti:

$$\mathbf{H}_{\text{out}} = \begin{bmatrix} \mathbf{h}_1 \\ \mathbf{h}_2 \\ \mathbf{h}_3 \\ \mathbf{h}_4 \end{bmatrix} \in \mathbb{R}^{4 \times 512}$$

Ciascun vettore $\mathbf{h}_i \in \mathbb{R}^{1 \times 512}$ attraversa la testa di predizione lineare finale ($\mathbf{W}_{\text{act}} \in \mathbb{R}^{512 \times 14}$, $\mathbf{b}_{\text{act}} \in \mathbb{R}^{14}$):

$$\hat{\mathbf{A}}_t = \mathbf{H}_{\text{out}} \mathbf{W}_{\text{act}} + \mathbf{B}_{\text{act}} = \begin{bmatrix} -0.120 & 0.450 & -0.890 & 1.200 & -0.050 & 0.310 & 0.000 & \dots & 0.00 \\ -0.118 & 0.455 & -0.887 & 1.204 & -0.048 & 0.312 & 0.050 & \dots & 0.05 \\ -0.115 & 0.461 & -0.882 & 1.210 & -0.045 & 0.315 & 0.120 & \dots & 0.12 \\ -0.110 & 0.470 & -0.875 & 1.218 & -0.040 & 0.320 & 0.200 & \dots & 0.20 \end{bmatrix} \in \mathbb{R}^{4 \times 14}$$

La prima riga comanda l'assetto articolare a $t + 20$ ms, mentre le righe successive delineano in modo coerente e continuo l'evoluzione del moto fino a $t + 80$ ms, eliminando alla radice ogni latenza di decodifica iterativa.

3 Action Chunking e Temporal Ensembling

La vera innovazione meccanica di ACT, che ha permesso di eliminare il *compounding error* superando i limiti del controllo a step singolo, è l'interazione tra Action Chunking e Temporal Ensembling a runtime.

3.1 Action Chunking (Mitigazione della miopia temporale)

Se il modello prevedesse 1 singolo passo per volta, a 50 Hz la distanza tra due passi sarebbe di 20 ms. L'errore stocastico su un singolo frame farebbe deragliare i giunti del robot.

Predicendo H passi simultaneamente (es. $H = 100$, coprendo ben 2 secondi di movimento nel futuro), il Transformer impone una fortissima ****coerenza cinematica interna****: i layer di attenzione del Decoder assicurano che la posizione prevista al passo $h = 50$ sia fluidamente raccordata al passo $h = 51$. Il rumore casuale su un singolo asse viene soppresso dalla struttura del blocco.

3.2 Temporal Ensembling (Fusione Esponenziale)

Se il robot si limitasse a eseguire il chunk per intero ($H = 100$) prima di elaborare un nuovo frame, si comporterebbe come una macchina ad anello aperto per 2 secondi, ignorando completamente i cambiamenti dell'ambiente (es. l'oggetto si muove).

ACT usa un approccio a orizzonte scorrevole continuo (*Receding Horizon*): 1. A ogni singolo istante t (es. ogni 20 ms), la telecamera scatta una foto. 2. La rete neurale calcola un **nuovo chunk intero** di H passi. 3. Di conseguenza, all'istante t esisteranno H predizioni diverse e sovrapposte calcolate negli istanti precedenti.

ATTENZIONE: Come fa il robot a decidere quale setpoint usare se ha 100 predizioni diverse per l'istante attuale?

La soluzione è il **Temporal Ensembling**. Ad ogni passo t , il comando effettivamente inviato ai giunti $\bar{\mathbf{a}}_t$ è una media pesata esponenzialmente di tutte le H predizioni storiche valide per quell'istante.

La formula rigorosa per l'azione eseguita al tempo t è:

$$\bar{\mathbf{a}}_t = \frac{\sum_{i=0}^{H-1} w_i \cdot \hat{\mathbf{a}}_t^{(t-i)}}{\sum_{i=0}^{H-1} w_i}$$

Dove:

- $\hat{\mathbf{a}}_t^{(t-i)}$ è l'azione per l'istante t che era stata predetta all'istante $(t - i)$.
- $w_i = \exp(-m \cdot i)$ è il peso esponenziale decrescente (con $m \geq 0$ parametro di decadimento).

Effetto Fisico: Un peso esponenziale alto ($m > 0$) dà massima autorità alla predizione calcolata nell'ultimo istante ($i = 0$), rendendo il braccio reattivo ai disturbi. Le predizioni più vecchie ($i \rightarrow H$) fanno da *ancora di stabilità (smoothing)*, filtrando matematicamente il rumore bianco ad alta frequenza e azzerando il *jerk* (derivate terze) dei motori.

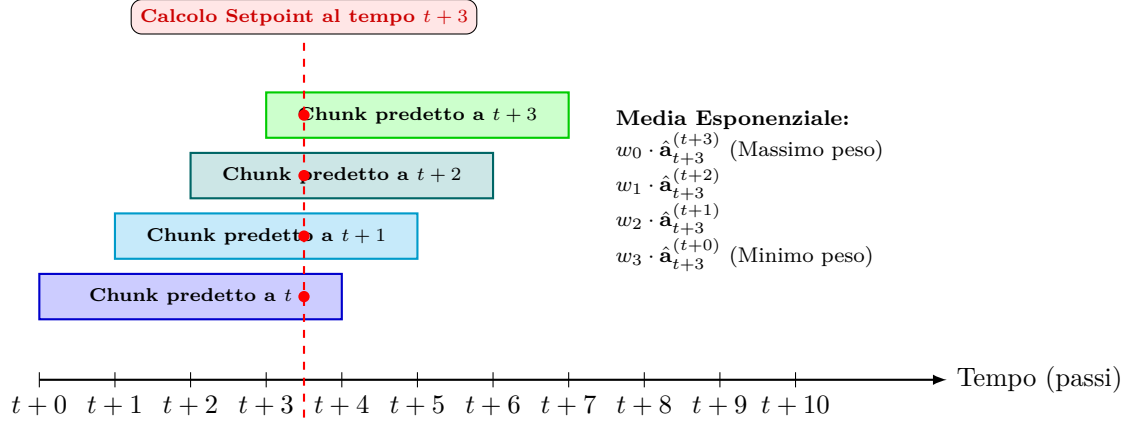


Figura 7: Rappresentazione del *Temporal Ensembling*. Ad ogni step temporale, il robot ricava il comando fisico interpolando verticalmente tutte le predizioni storiche sovrapposte che insistono su quel preciso istante. Il decadimento esponenziale w_i garantisce fluidità cinematica.

4 Addestramento: Funzione di Costo e Flusso dei Gradienti

L'architettura ACT è formalmente un *Conditional Variational Autoencoder* (CVAE). Di conseguenza, il suo addestramento è formulato come la massimizzazione dell'*Evidence Lower Bound* (ELBO), che nella pratica del Deep Learning viene capovolta e formulata come la minimizzazione di una funzione di costo congiunta.

Il modello possiede due insiemi di pesi addestrabili distinti:

- ϕ : I pesi del **CVAE Encoder** (strati di proiezione e Transformer Encoder latente).
- θ : I pesi della **Policy** (Transformer Vision Encoder, Transformer Decoder e Action Head).

La funzione obiettivo globale da minimizzare è una combinazione lineare di due forze contrapposte:

$$\mathcal{L}_{\text{ACT}}(\theta, \phi) = \mathcal{L}_{L1}(\theta, \phi) + \beta \mathcal{L}_{KL}(\phi)$$

dove β agisce da iperparametro di regolarizzazione (secondo il framework del β -VAE).

4.1 Le due componenti della Loss

1. Reconstruction Loss ($\mathcal{L}_{L1}$): Precisione Motoria Questa componente penalizza lo scostamento fisico tra la traiettoria predetta $\hat{\mathbf{A}}_t$ e quella reale eseguita dall'operatore umano $\mathbf{A}_t$. Invece dell'Errore Quadratico Medio (L2), che tende a generare traiettorie eccessivamente smussate, ACT utilizza la **L1 Loss (Errore Assoluto Medio)** per preservare l'alta frequenza e la nitidezza dei comandi motori:

$$\mathcal{L}_{L1}(\theta, \phi) = \sum_{h=1}^H \|\hat{\mathbf{a}}_{t+h} - \mathbf{a}_{t+h}\|_1$$

Cosa addestra? Il gradiente di questa loss ($\nabla \mathcal{L}_{L1}$) retropropaga attraverso l'intera rete. Aggiorna i pesi della Policy (θ) per insegnare al Decoder a ricostruire il movimento, ma, grazie al *Reparameterization Trick* ($\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$), il gradiente attraversa il nodo stocastico $\mathbf{z}$ e risale fino al CVAE Encoder (ϕ) addestrandolo. Questo insegna al CVAE a comprimere lo stile del movimento in un formato che sia effettivamente utile al Decoder per ridurre l'errore.

2. Kullback-Leibler Divergence ($\mathcal{L}_{KL}$): Regolarizzazione Latente Senza alcun vincolo, il CVAE Encoder potrebbe memorizzare arbitrariamente le traiettorie in punti sparsi dello spazio latente, rendendo impossibile campionare stili validi a runtime. La Divergenza KL costringe la distribuzione appresa dal CVAE, $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$, ad aderire a una distribuzione Prior neutra $\mathcal{N}(\mathbf{0}, \mathbf{I})$:

$$\mathcal{L}_{KL}(\phi) = D_{KL}(\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2) \parallel \mathcal{N}(\mathbf{0}, \mathbf{I})) = \frac{1}{2} \sum_{i=1}^{32} (\sigma_i^2 + \mu_i^2 - 1 - \ln(\sigma_i^2))$$

Cosa addestra? Questa loss dipende unicamente dalle uscite del CVAE. Pertanto, il suo gradiente ($\nabla \mathcal{L}_{KL}$) aggiorna **esclusivamente** i pesi ϕ dell'Encoder, forzando le medie μ_i verso lo zero e le varianze σ_i^2 verso 1. Un coefficiente β severamente calibrato previene il *posterior collapse*, costringendo il vettore $\mathbf{z}$ a mantenere l'intenzione modale dell'azione.

4.2 Il Paradosso Causale: Perché il CVAE scompare a Runtime?

Durante l'addestramento, il modello agisce come un Autoencoder: legge il futuro ($\mathbf{A}_t$), lo comprime in $\mathbf{z}$ e lo decomprime in $\hat{\mathbf{A}}_t$. Ma a runtime (inferenza), il robot si trova nel presente: **il chunk futuro $\mathbf{A}_t$ non esiste ancora**, poiché è esattamente ciò che il robot deve calcolare.

Di conseguenza, il **CVAE Encoder diventa fisicamente incalcolabile a runtime**. Deve essere rimosso.

L'impronta latente sulla Policy Finale Durante l'addestramento, il modello opera in modalità Autoencoder: legge l'azione futura $\mathbf{A}_t$ tramite il CVAE, la comprime in $\mathbf{z}$ e la decomprime in $\hat{\mathbf{A}}_t$ condizionandosi su quell'informazione privilegiata. Tuttavia, durante il deployment sul robot (inferenza), l'azione futura $\mathbf{A}_t$ non è ancora avvenuta: è esattamente ciò che la Policy deve generare basandosi solo sul presente.

Di conseguenza, l'ingresso al CVAE Encoder non esiste a runtime, rendendo il blocco matematicamente incalcolabile. Rimuovere l'impalcatura CVAE è quindi un vincolo causale imprescindibile. Se il CVAE scompare, a cosa è servito? La risposta risiede nell'**architettura dei pesi del Transformer Decoder (θ)**.

Il CVAE come Scultore di Mappe Neurali Multimodali Consideriamo uno scenario multimodale (un "bivio"): l'operatore umano, per aggirare un ostacolo, a volte passa a destra e a volte a sinistra. Senza il condizionamento stilistico del CVAE, il Transformer Decoder riceverebbe immagini identiche associate ad azioni opposte. La L1 Loss, nel disperato tentativo di minimizzare l'errore medio, costringerebbe il Decoder ad apprendere la media matematica delle due traiettorie, portando il braccio del robot a collassare e schiantarsi centralmente contro l'ostacolo (*Mode Collapse*).

Il CVAE risolve questo stallo. Durante il training, invia al Decoder un vettore $\mathbf{z}$ stocastico "alto" per l'azione "destra" e un vettore $\mathbf{z}$ "basso" per l'azione "sinistra". Il Decoder non apprende più un'azione media, ma **apprende una funzione condizionata**: scolpisce fisicamente nei suoi pesi e nelle sue matrici di attenzione dei percorsi neurali dedicati alle diverse modalità, separando l'intenzione modale.

Perché la Policy risponde dinamicamente a $\mathbf{z} = \mathbf{0}$ A runtime, forziamo deterministicamente $\mathbf{z}$ al vettore nullo ($\mathbf{z} = [0, 0, \dots, 0]$), il centro esatto del Prior stocastico. Questo potrebbe sembrare un input statico e "vuoto", ma per il Decoder che è stato addestrato sotto il condizionamento CVAE, $\mathbf{z} = \mathbf{0}$ non è un comando statico, ma una **"Chiave di Attivazione" della Modalità Nominale**.

Quando $\mathbf{z} = \mathbf{0}$ entra nel Decoder, esso attiva una proiezione lineare pre-addestrata (la matrice W_z) che "sintonizza" l'intero Transformer su una configurazione media-ponderata delle traiettorie umane di successo. **A questo punto, la scelta specifica del movimento è interamente delegata alla Cross-Attention Visiva:**

- Se l'ostacolo appare a destra, la Cross-Attention interroga la Memory visiva, e i percorsi neurali scolpiti dai pesi θ (sintonizzati da $\mathbf{z} = \mathbf{0}$) dirigono il movimento a sinistra.
- Se l'ostacolo appare a sinistra, gli stessi percorsi neurali, interrogando un contesto visivo diverso, dirigono il movimento a destra.

In sintesi, $\mathbf{z}$ ha fornito l'impalcatura temporanea per insegnare al Decoder che esistono traiettorie coerenti e non collassate. Rimuovendolo e fissandolo a zero, il robot esegue la traiettoria "nominale" appresa, ma conserva la coerenza cinematica multimodale impressa dall'impalcatura del CVAE, evitando le medie disastrose e rispondendo dinamicamente all'osservazione visiva.

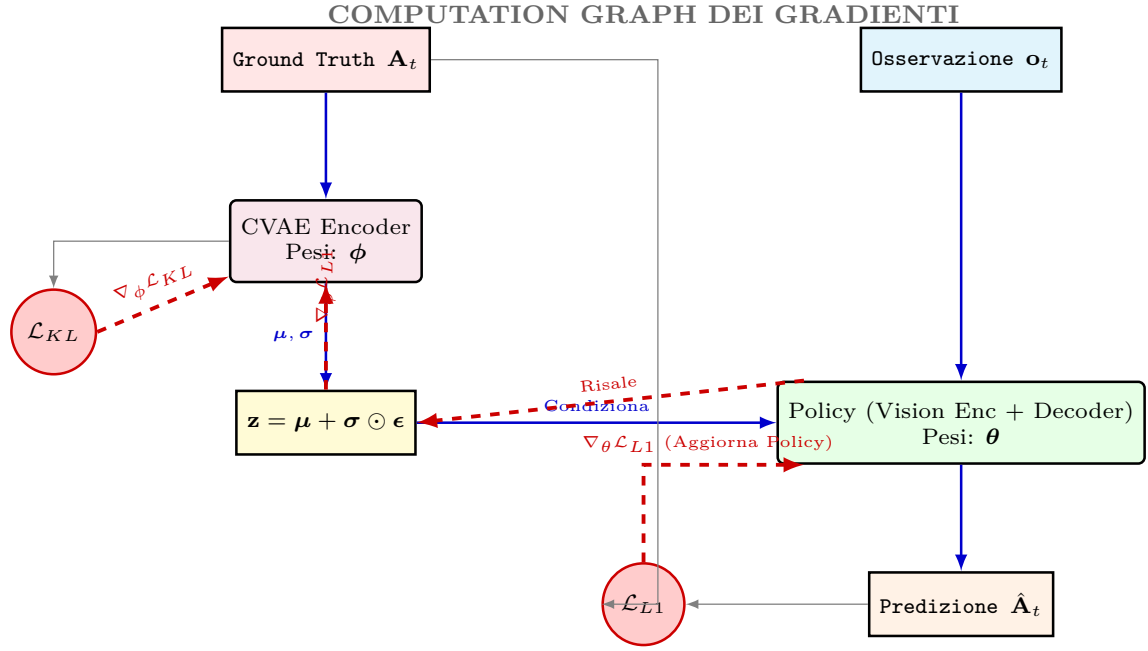


Figura 8: Flusso della *Backpropagation*. La **Policy** (θ) è addestrata esclusivamente dalla $\mathcal{L}_{L1}$ per minimizzare l'errore di imitazione. Il **CVAE** (ϕ), invece, riceve gradienti da due fonti: dalla $\mathcal{L}_{KL}$ (che comprime le distribuzioni verso il Prior) e dalla $\mathcal{L}_{L1}$ (che risale attraverso z grazie al *reparameterization trick*, forzando il CVAE a produrre stili latenti utili alla riduzione dell'errore).